



TECNOLOGIAS DE LA INFORMACIÓN

Evolución de los lenguajes de programación y transición hacia la POO

NOMBRE:

Dennis Leonardo Pacheco Álvarez

MATERIA:

PROGRAMACION ORIENTADA A OBJETOS (E)

NIVEL:

2do nivel

2025 - 2026



Introducción

Cuando hablamos de programación, muchas veces empezamos directamente con un lenguaje moderno como Java o Python sin saber de dónde vienen ni por qué son como son. Este informe trata sobre la historia y evolución de los lenguajes de programación, desde los más básicos y difíciles de usar, hasta llegar a lo que hoy conocemos como Programación Orientada a Objetos (POO).

Entender esta evolución me parece importante porque ayuda a comprender por qué la POO existe y qué problemas vino a resolver. No es solo historia: es la base para entender mejor cómo pensamos cuando programamos hoy en día.

Evolución de los Lenguajes de Programación

- **Los lenguajes de bajo nivel: cuando las máquinas mandaban**

Los primeros lenguajes de programación eran básicamente instrucciones directas para el hardware. El más básico de todos es el lenguaje máquina, que consiste en secuencias de ceros y unos (código binario). Imagina tener que escribir algo como 01001000 01100101 solo para que la computadora haga una operación simple: eso era programar en los años 40 y 50.

Luego apareció el lenguaje ensamblador (Assembly), que fue un avance importante porque permitía usar palabras cortas como MOV, ADD o SUB en lugar de puros números. Aunque seguía siendo complicado, ya era más legible para los programadores. El problema era que estas instrucciones dependían totalmente del tipo de procesador, o sea, un programa hecho para una máquina no servía en otra



- **Los lenguajes de alto nivel: programar más como humanos**

A finales de los años 50 y durante los 60, surgieron los lenguajes de alto nivel. El primero

grande fue FORTRAN (1957), diseñado para cálculos matemáticos y científicos. Luego vino COBOL (1959), pensado para negocios y manejo de datos. Más adelante llegaron BASIC, Pascal y otros que hicieron la programación mucho más accesible.

La gran ventaja de estos lenguajes era que se parecían más al inglés cotidiano. En lugar de instrucciones en binario o comandos crípticos, podías escribir cosas como PRINT "Hola" o IF x > 5 THEN... Lo que uno escribía lo traducía un programa llamado compilador al lenguaje que la máquina entendía

El Paradigma Procedural y sus Limitaciones

Con la llegada de lenguajes como C, Pascal y FORTRAN, se consolidó el llamado paradigma procedural o estructurado. En este modelo, los programas se organizan como una serie de pasos (procedimientos o funciones) que se ejecutan uno tras otro. Es como una receta: haz esto, luego esto, y si pasa tal cosa, haz aquello.

Por mucho tiempo funcionó bien, pero a medida que los programas crecían en tamaño y complejidad, empezaron a aparecer problemas serios:

- El código se volvía difícil de leer y mantener. Cuando un programa tiene miles de líneas con variables globales y funciones que se llaman entre sí de manera complicada, encontrar un error se convierte en una pesadilla.



- Era muy difícil reutilizar el código. Si necesitabas algo similar en otro programa, casi siempre tenías que copiar y pegar, lo que generaba duplicación y más posibilidades de errores.

El software era poco modular. Era complicado dividir el programa en partes independientes que pudieran trabajar sin afectarse mutuamente, lo que hacía el trabajo en equipo mucho más difícil.

Estos problemas se hicieron más evidentes en los años 70 y 80, cuando las empresas empezaron a necesitar sistemas de software más grandes y complejos. Fue entonces cuando la comunidad de programadores empezó a buscar una solución mejor.

El Surgimiento de la Programación Orientada a Objetos

La idea de la POO no apareció de la noche a la mañana. Simula, creado en Noruega en los años 60, fue el primer lenguaje en introducir el concepto de "objetos" y "clases". Luego, en los 70, Smalltalk desarrolló esta idea más a fondo. Pero fue en los años 80 y 90, con lenguajes como C++ y Java, cuando la POO se convirtió en el estándar de la industria.

La idea central de la POO es organizar el código alrededor de objetos, que son representaciones de cosas del mundo real (o abstractas) que tienen tanto datos (atributos) como comportamientos (métodos). Por ejemplo, en lugar de tener funciones separadas y variables sueltas para manejar un estudiante, creamos un objeto Estudiante que tiene su nombre, su edad, y puede hacer cosas como inscribirse o calcular su promedio

Comparación: Procedural vs. Orientada a Objetos

Para entender mejor las diferencias, aquí un resumen de los puntos más importantes:



Organización del código:

- Procedural: el código se divide en funciones o procedimientos que operan sobre datos separados.
- POO: el código se agrupa en clases que combinan datos y comportamiento en una sola unidad.

Reutilización:

- Procedural: para reutilizar código hay que copiar funciones, lo que puede generar inconsistencias.
- POO: la herencia permite que una clase tome características de otra, favoreciendo la reutilización sin duplicar código.

Mantenimiento:

- Procedural: cambiar una parte del sistema puede romper otras partes, especialmente con variables globales.
- POO: el encapsulamiento protege los datos internos de un objeto, reduciendo el impacto de los cambios.

Modularidad:

- Procedural: la modularidad existe (mediante funciones y archivos), pero las dependencias entre partes pueden ser difíciles de controlar.

POO: las clases y objetos son módulos naturales, más fáciles de probar y mantener de forma independiente





Reflexión Personal

Honestamente, antes de investigar esto, nunca me había preguntado por qué programamos en Java y no en algo parecido al lenguaje máquina. Simplemente lo tomaba como un hecho. Pero entender la historia cambia la perspectiva completamente.

Me llama la atención cómo cada avance en los lenguajes de programación vino de una necesidad real: los lenguajes de alto nivel aparecieron porque el ensamblador era demasiado difícil; la POO apareció porque los programas procedurales se volvieron imposibles de manejar. Es una enseñanza importante: en tecnología, las soluciones no caen del cielo, surgen de los problemas concretos que la gente enfrenta.

Para mí, como estudiante de primer semestre aprendiendo Java, saber que la POO fue diseñada para resolver problemas reales me motiva a aprenderla en serio. No es solo una forma rara de organizar código; es una manera de pensar que ha probado funcionar en proyectos grandes del mundo real. Comprender de dónde viene hace que quiera aprovecharla bien.

Conclusión

La evolución de los lenguajes de programación es una historia de soluciones progresivas. De los ceros y unos del lenguaje máquina, pasamos al ensamblador, luego a los lenguajes de alto nivel, y finalmente a la Programación Orientada a Objetos. Cada paso respondió a limitaciones reales del modelo anterior.

El paradigma procedural fue un gran avance pero tenía problemas de organización, reutilización y mantenimiento que se hicieron críticos cuando los programas crecieron en tamaño.





La POO ofreció una solución más natural y escalable, modelando el software como objetos que interactúan entre sí.

Comprender esta historia no es solo cultura general: es entender el "por qué" detrás de las herramientas que usamos. Y eso, al final, hace mejor a cualquier programador.

Bibliografía y Referencias

Ribera-Carbonell, E., & Arias-Robles, F. (2025). Evolución del uso de lenguajes de programación y herramientas digitales en el periodismo de datos español. [Evolution of the use of programming languages and digital Tools in Spanish data journalism] *Estudios Sobre El Mensaje Periodístico*, 31(1), 253-267. <https://doi.org/10.5209/emp.96796>

Milton Rafael, V. P., Joofre Antonio, H. T., Antonio Steeven Gómez Moreno, & Luis Fernando Vines Sánchez. (2018). COMPARACIÓN DE TENDENCIAS TECNOLÓGICAS EN APLICACIONES WEB. *3C Tecnología*, 7(3), 28-49.

<https://www.proquest.com/scholarly-journals/comparación-de-tendencias-tecnológicas-en/docview/2119298245/se-2>

Epitech. (s. f.). *Evolución de los lenguajes de programación: Inicio y actualidad*. Epitech - Escuela Superior de Informática. <https://www.epitech-it.es/evolucion-lenguajes-de-programacion/>



Anexos

[◀ Volver a la página Resultados](#) 1 de 2.119 [8 Texto completo](#) | Revista científica

Evolución del uso de lenguajes de programación y herramientas digitales en el periodismo de datos español

Alternate title: Evolution of the use of programming languages and digital Tools in Spanish data journalism

Ribera-Carbonell, Enrique; Arias-Robles, Félix. [Estudios Sobre el Mensaje Periodístico](#); Madrid Tomo 31, N.º 1, (2025): 253-267, DOI:10.5209/emp.96796

PDF Resumen/detalles

1 de 16 Anchura de la página

Evolución del uso de lenguajes de programación y herramientas digitales en el periodismo de datos español

Enrique Ribera-Carbonell
Universidad Miguel Hernández de Elche (UMH)

Félix Arias-Robles
Universidad Miguel Hernández de Elche (UMH)

<https://dx.doi.org/10.5209/emp.96796> Recibido: 30 de junio de 2024 / Aceptado: 31 de octubre de 2024

Resumen. El periodismo de datos es una de las especialidades con mayor proyección en los medios de

← → ↺ epitech-it.es/evolucion-lenguajes-de-programacion/

88

Agenda Blog

EVOLUCIÓN DE LOS LENGUAJES DE PROGRAMACIÓN: INICIO Y ACTUALIDAD

Tabla de contenidos

Cuando hablamos de lenguajes de programación, parece que estemos refiriéndonos a algo moderno, algo actual pero, lo cierto es que el primer lenguaje de programación se creó hace más de dos siglos. No te lo esperabas, ¿Verdad? En este post hacemos un repaso de la historia de los lenguajes de programación, desde el primero hasta los más actuales y los lenguajes que más se utilizan ¡Continúa leyendo y descúbrelos!

Historia de los lenguajes de programación

El primer lenguaje de programación

En 1801, un inventor francés, Joseph Marie Jacquard fue el inventor del telar programable. Jacquard creó unas tarjetas programables que se introducían en un telar que leía el código, que contenía las instrucciones y de esta forma, se automatizaba el proceso.

<https://www.epitech-it.es/event-directory/>